Bomb Lab: gdb disassembly 분석

전체 지도 → 어셈블리 읽는 법 → 실제 추적 시연 → 자주 틀리는 읽기 → gdb 확인 · 시험 직전 암기표 · phase별 풀이는 문제 풀이의 Bomb Lab 섹션

1 폭탄의 뼈대와 phase 지도

bomb는 `read_line`으로 한 줄을 읽고 `phase_N`을 부르고 `phase_defused`로 넘어가는 루프다. 입력이 조건과 다르면 `explode_bomb`이 호출된다 — disassembly에서 이 call이 보이면, 그 직전의 `cmp/jX`가 그 phase의 통과 조건이다. 어떤 phase든 읽는 순서는 같다: 입력이 어디에 깔리는지 → 조건을 검사하는 cmp/jX → 통과 경로.

phase	입력	핵심 장치	읽기 출발점	gdb 확인
1	문자열 한 줄	`strings_not_equal` — 같으면 %eax=0	`mov $ADDR,%esi`의 ADDR이 정답 문자열	`x/s ADDR`
2	정수 6개	`read_six_numbers` → `(%rsp)`~`0x14(%rsp)` · 현재 = 2 × 이전	루프 안의 `cmp/je` (§2 추적 시연)	`x/6dw $rsp`
3	정수 2개	switch = jump table · 첫 검사 `cmpl $0x7; ja`는 unsigned	`jmp *ADDR(,%rax,8)`의 ADDR이 테이블 주소	`x/8gx ADDR`
4	정수 2개	`func4` 이중 재귀: f(n)=f(n−1)+f(n−2)+x	base case(n≤0→0, n==1→x)부터 C로 옮기기	반환값 `%rax` 추적
5	정수 2개	x1의 하위 4 bit가 시작 인덱스 · arr[i] 체이싱 15회 · 지나온 값의 합 = x2	배열 16개	`x/16dw ARR`
6	1~6 순열	연결 리스트 재배열 · 노드 +0 value, +4 index, +8 next	`mov 0x8(%rdx),%rdx`가 next를 읽는 명령	`x/2wx NODE` · `x/gx NODE+8`

각 phase의 전체 disassembly 분석은 문제 풀이에 있다. 이 표의 모든 행은 아래 §2의 규칙으로 읽는다.

2 어셈블리 읽기 — 랩이 전제하는 배경

핸드아웃은 "gdb로 disassembly를 읽어라"까지만 말하고 아래는 전부 전제한다. 여기가 막히면 phase가 어려운 게 아니라 asm이 안 읽히는 것. 규칙을 읽은 뒤 이어지는 추적 시연에서 실제 적용을 확인한다.

어셈블리 읽기 규칙 (lec4·lec5)

- AT&T 문법: `mov src, dst` (Intel과 반대 방향). `(%rax)`는 메모리 역참조, `lea`는 주소 계산만 하고 메모리는 읽지 않음. 접미사 b/w/l/q = 1/2/4/8바이트
- `cmp b, a`는 a − b를 계산해 flag만 세팅한다. 비교 결과를 저장하거나 점프하지는 않음
- 실제 판단은 다음 `jX`가 그 flag를 해석해 한다. `jle`면 "a ≤ b". 기준은 두 번째 피연산자 a
- `ja/jbe`는 unsigned 비교, `jg/jle`는 signed 비교. phase 3 첫 검사 `cmpl $0x7; ja`가 unsigned라 음수 입력은 7보다 큰 것처럼 보여 바로 폭발
- `D(%rb,%ri,s)` = Mem[%rb + %ri×s + D]. jump table의 `jmp *0x402540(,%rax,8)`도 이 형태

함수 호출 규약 (lec6)

- 인자는 `%rdi %rsi %rdx %rcx %r8 %r9` 순서, 반환값은 `%rax`. `call sscanf` 직전에 rsi·rdx에 뭘 싣는지가 입력 형식 힌트
- `sscanf` 반환값 %eax = 읽은 항목 수 → `cmp $0x1,%eax; jg` = "2개 이상 입력해야 통과" (phase 3·4)
- `read_six_numbers` : 정수 6개가 %rsp부터 int 배열로 깔림 → `(%rsp)`, `0x4(%rsp)` … `0x14(%rsp)` (phase 2·6)
- 이 과목의 `func4`는 인자 2개(n=%edi, x=%esi)의 이중 재귀: n≤0 → 0, n==1 → x, 아니면 func4(n−1,x)+func4(n−2,x)+x (lea (%rax,%rbp,1),%r12d가 첫 재귀 결과에 x를 더함 → f(1)=x, f(2)=2x, f(3)=4x, f(4)=7x, f(5)=12x). base case부터 읽고 C로 옮긴 뒤 원하는 반환값이 나오는 입력을 역산 (phase 4)

자료구조와 메모리 (lec7)

- phase 1: `strings_not_equal`은 같으면 %eax=0. 정답 문자열 주소는 `mov $ADDR,%esi`로 실려 있음 → `x/s ADDR`로 바로 읽힘
- phase 3: switch = jump table. `x/8gx TABLE`로 case 주소 8개를 읽고 case별 상수 연산 추적
- phase 5: 입력의 하위 4 bit가 시작 인덱스. arr[i]를 다음 인덱스로 삼아 15번 따라가는 포인터 체이스. 배열 16개는 `x/16dw`
- phase 6 노드: value·index는 +0/+4의 int, next는 +8의 8바이트 포인터. 다음 노드로 갈 때 주소에 8을 더하는 게 아니라 +8에 저장된 주소를 읽는 것

추적 시연 — 위 규칙을 실제 코드에 적용: phase 2 루프

문제 페이지 phase 2의 반복 부분이다. 입력 `1 2 4 8 16 32`가 `(%rsp)`부터 4바이트씩 깔린 상태에서, %rbx는 두 번째 정수 `0x4(%rsp)`를 가리키고 %rbp는 마지막 정수 다음을 가리킨다.

```
mov  -0x4(%rbx), %eax   # 이전 원소를 읽음
add  %eax, %eax          # 2배로 만듦
cmp  %eax, (%rbx)        # (현재 원소) - %eax 로 flag 세팅
je   <phase_2+41>       # 같으면(ZF=1) 아래 call을 건너뜀
call <explode_bomb>      # 다르면 폭발
add  $0x4, %rbx          # 다음 정수로
cmp  %rbp, %rbx
jne  <phase_2+27>       # 끝이 아니면 반복
```

이 코드가 실제로 쓰는 것만 추적한다 — 레지스터 %eax·%rbx·%rbp, 스택의 정수 6개, flag ZF.

회차	mov → %eax	add → %eax	(%rbx)	cmp 결과 → ZF	판정
1	이전 원소 1	2	2	2 − 2 = 0 → ZF=1	je 통과, 계속
2	이전 원소 2	4	4	4 − 4 = 0 → ZF=1	je 통과, 계속
…	8, 16, 32도 같은 검사를 통과하면 %rbx = %rbp가 되어 jne 불통과 → 루프 종료, 해제				

여기 쓰인 규칙은 전부 위 박스의 것이다: `cmp`는 두 번째 피연산자 기준(현재 − 이전×2), `je`는 ZF=1일 때 점프, `-0x4(%rbx)`는 D(%rb) 주소 계산, 정수 6개의 스택 배치는 `read_six_numbers` 규칙. 어느 하나라도 2배가 아니면 je가 불통과라 폭발 — 이것이 "현재 = 2 × 이전" 조건의 코드상 모습이다. 같은 읽기를 phase 전체에 반복하는 것이 이 랩의 핵심이다.

3 자주 틀리는 읽기 (수정 메모)

처음에 잘못 읽었다가 고친 것 — §2 규칙이 실제로 꼬이는 지점

- `mov 0x30(%rsp,%rax,4), %eax` → 주소 자체가 아니라 Memory[%rsp + 48 + 4×%rax]의 값을 읽음
- `cmp %eax, 0x0(%rbp)` → %rbp는 입력값이 아니라 입력값의 주소. 입력값끼리 비교하는 것
- `cmp $0x5, %ebx; jle` → %ebx ≤ 5이면 점프. 점프 주소(0x401145)와 값을 비교하는 게 아님
- `mov 0x8(%rdx), %rdx` → node.next (포인터). %rdx에 8을 더하는 게 아님
- `cmp %eax, (%rbx); jle` → 현재 노드 value ≤ 다음 노드 value

4 gdb로 직접 확인

gdb 명령

- `break explode_bomb` : 함수 진입 지점에 breakpoint를 걸어 실행을 멈춘다. continue하면 함수 실행이 재개된다. 실제 동작과 외부 기록 여부는 이 바이너리의 함수 본문 미확인 — "멈췄으니 안전"이 아니라 진입 시점에 멈춰 조사하는 용도 · `disas` (함수 disassemble) · `stepi` / `nexti` (한 명령씩)
- `x/s ADDR` : 주소의 문자열 읽기 (phase 1) · `x/8gx ADDR` : 8바이트씩 8개 (jump table) · `x/16dw ADDR` : 4바이트 10진수 16개 (배열) · `info registers`

- asm을 볼 때 순서: (1) 인자가 어떤 register인지 (2) 비교·점프로 만들어지는 구조 (3) 메모리 접근의 offset과 scale. 순서대로 적고 C로 옮기기
- `break explode_bomb`에 걸어 두고 `stepi`로 한 줄씩 진행하면서, §2 추적 표의 %eax·%rbx·ZF가 `info registers`와 `x/6dw $rsp`에 그대로 찍히는지 대조해 볼 것 — 표를 손으로 재현하는 것이 가장 빠른 확인

5 강의 연결 — 놓치기 쉬운 포인트

- lec4 addressing mode의 `D(%rb,%ri,s)` 계산식 → phase 3 테이블 주소와 phase 5 배열 접근이 전부 이 형태
- lec5 condition codes: `ja`는 CF=0·ZF=0일 때(unsigned "위"), `jg`는 ZF=0·SF=OF일 때(signed "큼") 점프. 같은 `cmp` 결과라도 쓰인 jump에 따라 해석이 달라짐 → phase 3·6 검사 읽기의 핵심

- lec6 스택 프레임: caller가 인자 세팅 → call → callee가 필요한 만큼 `sub $N,%rsp` 로 프레임 확보. 모든 함수가 만드는 건 아님 — 다른 함수를 부르지 않는 함수(leaf) 등은 프레임 없이 끝낼 수 있음. func4 재귀 프레임을 손으로 한 번 그려볼 것
- lec7 struct 정렬: 포인터는 8바이트 정렬 → int 2개(8바이트) 뒤 포인터는 +8. phase 6 노드 레이아웃이 그대로
- §3의 실수 5개 + mov/lea 혼동 + cmp 방향 혼동 — 분석할 때 실제로 틀린 유형이라, 시험 전에 한 번 더 확인해 둘 것

6 외워야 할 것 (시험 직전 체크)

- 인자 레지스터 순서: **%rdi → %rsi → %rdx → %rcx → %r8 → %r9**, 반환값 **%rax**
- callee-saved(함수가 사용·변경했다면 원래 값을 보존해야 하는): `%rbx %rbp %r12–%r15` · caller-saved(호출 뒤 날아갈 수 있는): `%rax %rcx %rdx %rsi %rdi %r8–%r11`
- 스택은 낮은 주소로 자람 · `call` = 리턴 주소 push 후 점프, `ret` = pop 후 점프
- gdb 최소 세트: `b explode_bomb` (함수 진입 시 정지 · continue하면 실행 재개) · `disas 함수명` · `stepi` / `nexti` · `info registers` · `x/s` 문자열 · `x/6dw` 정수 6개 · `x/8gx` 테이블 · `finish` 함수 끝까지
- 비교 읽기: `cmp` 는 flag를 세팅만 하고, 실제 판단은 다음 `jX` 가 그 flag를 해석. `cmp $imm, reg; jX` → "reg X imm"은 jX의 부호 의미(signed/unsigned)가 맞을 때만 성립하는 축약

7 정답 모음

– 정답 보기

```
Phase 1: Current frameworks are mainly designed for a solo training case.
Phase 2: 1 2 4 8 16 32
Phase 3: 4 0
Phase 4: 24 2
Phase 5: 5 115
Phase 6: 4 2 3 6 1 5
```